
Project 1 Report

STU FIIT



Adrian Quirante González
Lucas Daniel Espitia Corredor

Neural Networks (ZNEUS)

09-11-2025

Experiment 1 - Baseline

For the first experiments, we used the baseline parameters, which are the default values of our Config.

Listing 1: Baseline parameters

```
#This are the hyperparams
@dataclass(frozen=True)
class Config:
    # --- ARCHITECTURE ---
    input_dim: int = 15
    output_dim: int = 1
    hidden_layers: Tuple[int, ...] = (1,)
    activation: str = 'relu'
    weight_init: str = 'uniform'
    use_batch_norm: bool = False
    dropout_rate: float = 0.0
    skip_connections: bool = False

    # --- OPTIMIZATION ---
    optimizer: str = 'sgd'
    learning_rate: float = 1e-3
    momentum: float = 0.9
    beta1: float = 0.9
    beta2: float = 0.999
    epsilon: float = 1e-8
    batch_size: int = 32
    epochs: int = 50
    gradient_clipping: float = 0.0
    scheduler: str = 'None'

    # --- REGULARIZATION ---
    L1_lambda: float = 0.0
    L2_lambda: float = 1e-4
    early_stopping_patience: int = 5

    # --- LOSS FUNCTION ---
    loss_function: str = 'binary_cross_entropy'

    # --- RUN NAME ---
    run_name: str = 'Experiment1'
```

For the baseline, we have used a very basic network. It consists on one neuron as a hidden layer and one neuron as output. For the other parameters, we have decided to put its common initialization parameters, or simply not activating them if they were regularization, optimization or advanced parameters.

The relevant insights for this experiment come from analyzing the following values:

- F1 score max value: 0.44 - Accuracy max value: 0.84

This is why we selected f1 instead of accuracy. As you can see, we obtained 84% of good predictions, the exact number that we talked about in the previous section (code). The model is that simple that probably only predicts the big class.

F1 value isn't that bad considering it is a baseline model very simple. However, it can improve a lot, and that's what we want with the parameter search.

	Actual	Predicted	nPredictions
1	0	0	7750
2	0	1	250
3	1	0	1390
4	1	1	210

Figura 1: Confussion Values Experiment 1

We can see that the around 95% of predictions goes to class 0, due to imbalance. That's why we have 84% of accuracy but 40% f1 score. As we can see in Figura 2, we have a considerable quantity of false positives and false negatives.

In fact, what happens the most is those false negatives, where the model predicts 0 and the actual value is 1.

Taking all this into account, we now want to make our model more complex. Thus, what we are going to do is add more hidden layers.

Experiment 2

What happend here is that the model predicted only the 0 class. Thus, accuracy 83% (as comented) but f1 was 0. The loss was greater also. This is difficult to interpet, but out diagnostic is that we augmented complexity without regularizing well. For this reason, we are going to add batch_normalization to see if it helps.

Experiment 3

As we can see in Figure 3, with this change, loss improves to be like the baseline, but oscilating around the minimum. However, model continues to predict only the 0 class. This can be seen by looking at the logits of out training:

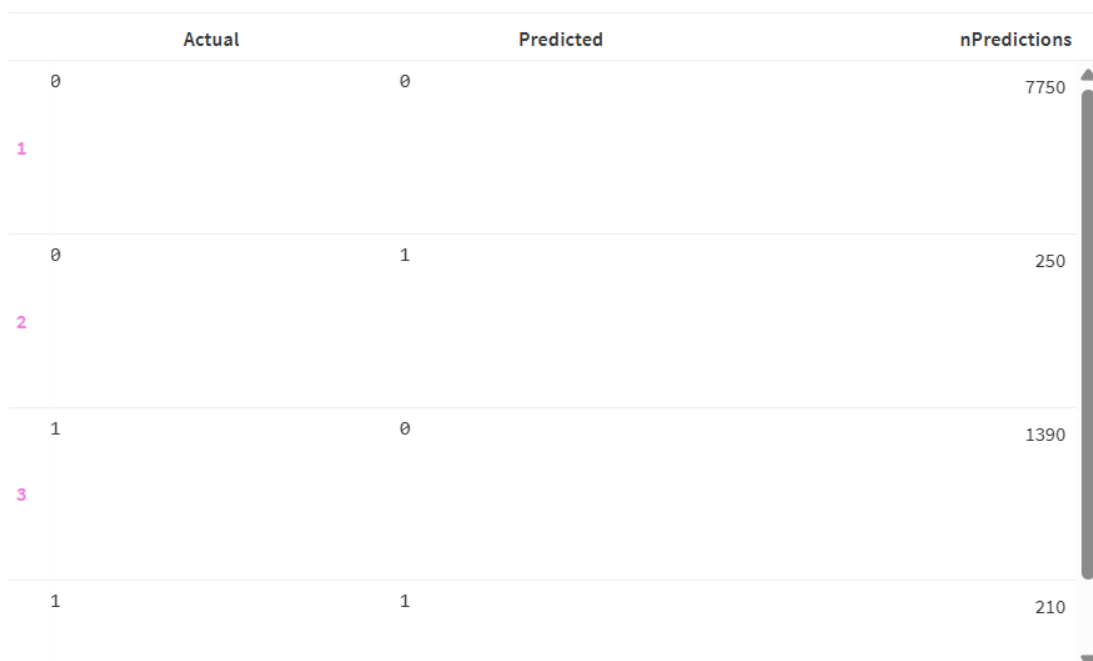


Figura 2: Confussion Matrix Experiment 1

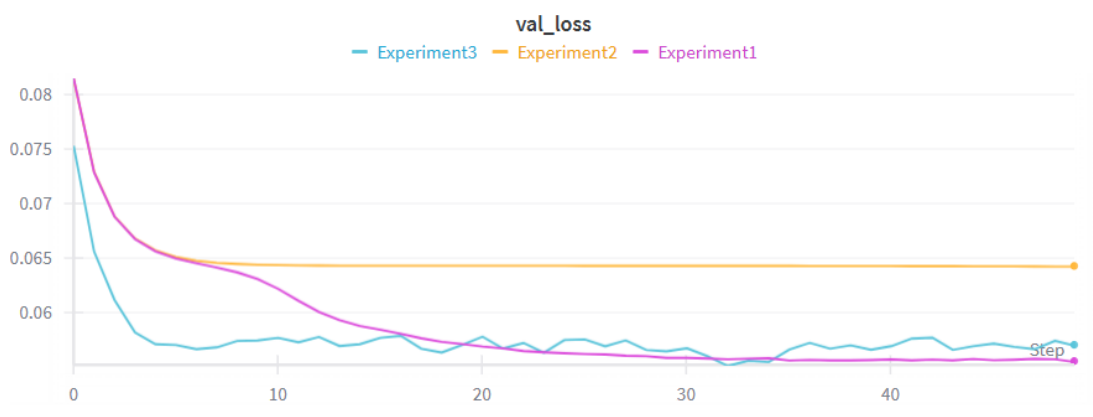


Figura 3: Loss tracking of first three experiments

Listing 2: Baseline parameters

```
Epoch [48/50] - Train Loss: 0.3393 - Val Loss: 0.0535 - Acc: 0.8333 Prec
: 0.0000 Rec: 0.0000
logits: min -6.2078, max -0.2965, mean -3.1032, std 1.9384
logits: min -5.0174, max -0.2965, mean -2.3548, std 1.4521
logits: min -6.1800, max -0.2965, mean -2.4986, std 1.8565
logits: min -5.9716, max -0.2965, mean -2.2726, std 1.3748
logits: min -6.4728, max -0.2965, mean -2.4857, std 1.5272
logits: min -5.5509, max -0.2965, mean -2.8077, std 1.7980
Epoch [49/50] - Train Loss: 0.3405 - Val Loss: 0.0531 - Acc: 0.8333 Prec
: 0.0000 Rec: 0.0000
logits: min -7.0492, max -0.2754, mean -3.3452, std 2.2567
logits: min -5.2708, max -0.2754, mean -2.4258, std 1.6813
logits: min -7.1457, max -0.2754, mean -2.6156, std 2.1225
```

```

logits: min -6.8379, max -0.2754, mean -2.3231, std 1.5535
logits: min -7.3865, max -0.2754, mean -2.5850, std 1.7784
logits: min -6.3400, max -0.2754, mean -2.9238, std 1.9935
Epoch [50/50] - Train Loss: 0.3362 - Val Loss: 0.0532 - Acc: 0.8333 Prec
: 0.0000 Rec: 0.0000

```

As we can see all logits are negative because of class imbalance, so all predictions are class 0. In the beginning our model was simpler, but now that it is complex we need to penalize more predicting that class when it is not. Because of that we are going to add a penalty in the loss, in order to care more about the positive class.

Experiment 4

Adding penalization in the loss caused immediate effect. Now our model does not only predict class 0. We got a maximum 0.47 f1 score, which is an improvement respect to the baseline. Accuracy dropped to 75%, which is normal as now it does not only predict class 0.

I see a problem, all maximum results were achieved in initial ephocs, and then it continued without finding something better. Becuase of that, we should change the patience implementation, as it is not working well.

On the other hand, we are going to continue adding complexity to the model. For now, we are going to add more neurons and hidden layers and we'll think of regularizing later.

Experiment 5

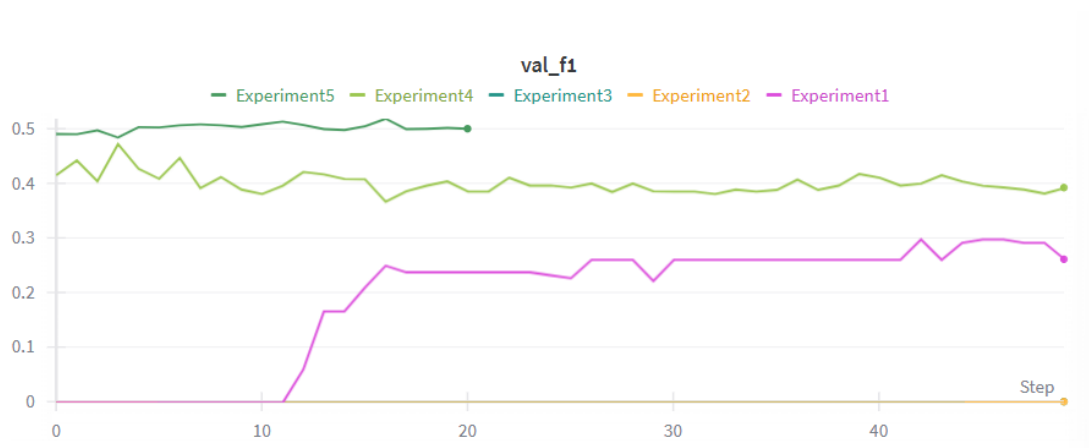


Figure 4: F1 Tracking of first 5 experiments

As we can see in Figure 4, we have improved performance by putting more complexity into the model. We'll continue this way, till we have to regularize the model and add optimizers. Moreover, it seems early stopping know works correctly, as it stopped around epoch 2 because it did not found any improvement.

However, we can see that the model loss increased a lot, as it is shown in Figure 5.

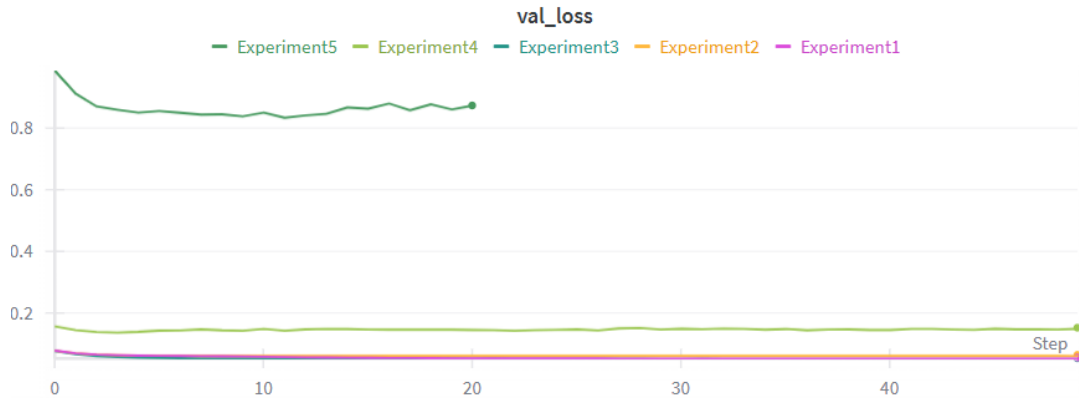


Figure 5: Loss tracking of first 5 experiments

While someone can think of this the bad way, we have to take into account that we have added a penalty in the loss since Experiment 5. This penalty causes an increase in loss, so we are not going to worry for now, but we are going to take this into account.

In the next experiment, we are going to keep adding complexity by doing our network deeper. Next network will have 16 hidden layers of size 8.

Experiment 6

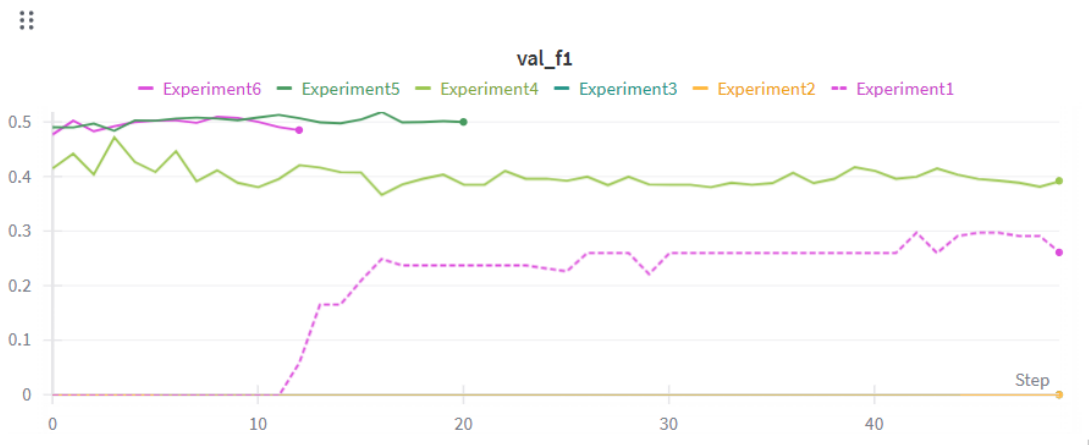


Figure 6: Validation f1 Experiment 6

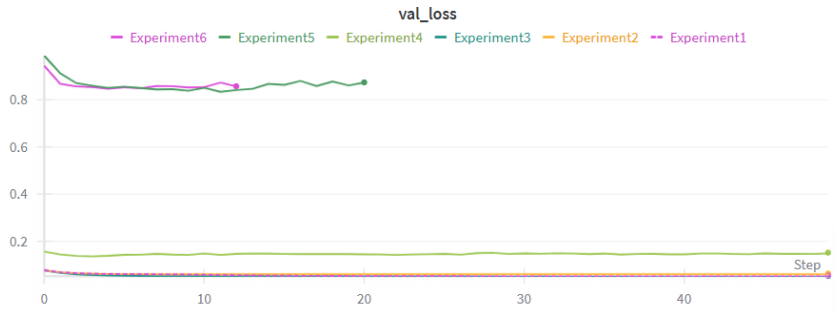
By looking at experiment 6, we can see that that we did not improved Experiment 5 f1. This is curious as we have done the model twice its size. In figure 6 we see some things that could explain these behaviour.

By comparing Trainig and Validation Loss, we can see how train loss has decreased with respect to Experiment 5, but validation loss has increased. This could be caused because our model is getting deeper but stays narrow. This can cause loss of information between layers.

For this reason, the next experiment will add skip connections, in order to pass information



(a) Training Loss



(b) Validation Loss

Figure 7: Experiment 6 Training vs Validation Loss

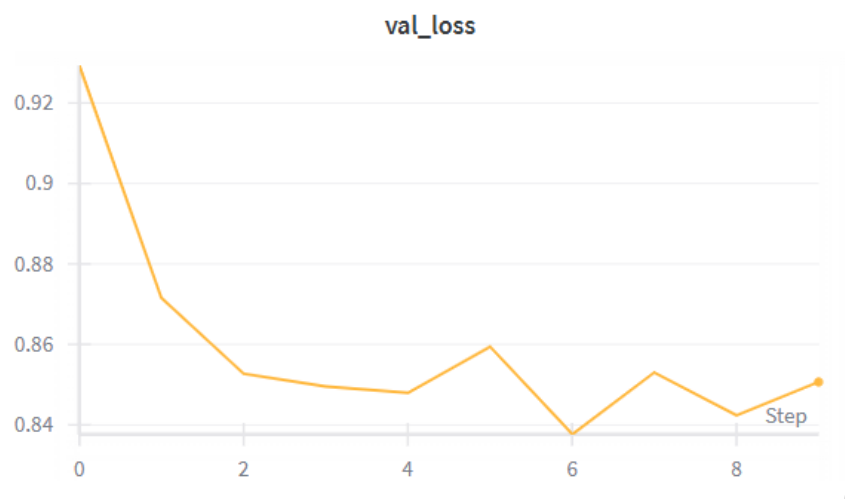
from previous layers to the deeper ones. We'll see how the results are now.

Experiment 7

This change did not improve anything. Results were pretty close to the ones of Experiment 6. At this point, we are not really sure of what to do to improve the model.



(a) Training Loss



(b) Validation Loss

Figure 8: Experiment 7 Training vs Validation Loss